

Procédure de supervision

Plateforme MSPR HealthAI Coach (MSPR3 / TPRES601).

Ce document est la procédure d'exploitation de la supervision : comment lancer la stack d'observabilité, ce qu'elle surveille, comment lire les alertes et quoi faire quand l'une d'elles se déclenche. La documentation technique détaillée (composants, versions, liste exhaustive des métriques) reste dans

`monitoring/README.md`.

1. Objectif et périmètre

L'objectif est de détecter rapidement une panne ou une dérive de la plateforme et de disposer des éléments (métriques, logs) pour diagnostiquer.

La supervision couvre les 8 services de la plateforme et l'hôte qui les héberge :

- les conteneurs applicatifs (API, AUTH, AI-Nutrition, Reco-Fitness, ETL, Front),
- les bases de données (PostgreSQL métier, PostgreSQL auth, MongoDB),
- l'hôte Docker (CPU, RAM, disque, charge),
- les logs de tous les conteneurs, centralisés.

La supervision est fournie comme un overlay Docker Compose (`docker-compose.monitoring.yml`) qui s'ajoute à la stack applicative sans la modifier. Elle est facultative : l'application fonctionne sans, mais on la lance dès qu'on veut observer ou exploiter la plateforme.

2. Ce qui est surveillé

2.1 Métriques applicatives (`/metrics`)

Quatre services exposent leurs métriques, collectées par Prometheus :

Service	Cible (job)	Endpoint
API Spring Boot	<code>api</code>	<code>/api/actuator/prometheus</code> (Actuator / Micrometer)
AI-Nutrition (FastAPI)	<code>ai-nutrition</code>	<code>/metrics</code>
Reco-Fitness (FastAPI)	<code>reco-fitness</code>	<code>/metrics</code>
AUTH (Hono / Bun)	<code>auth</code>	<code>/metrics</code>

On y trouve le débit, la latence et les codes de retour HTTP par route (modèle RED), plus la JVM (heap, GC, threads, pool de connexions) pour l'API.

Ces métriques ne sont visibles qu'après reconstruction des images des services concernés (l'instrumentation vit dans le code source). Tant qu'une image n'est pas à jour, la cible reste vue comme injoignable par Prometheus, sans impact sur les autres.

2.2 Conteneurs (cAdvisor)

cAdvisor (job `cadvisor`) expose, par conteneur, la consommation CPU, mémoire, réseau et entrées/sorties disque. C'est la source des deux alertes de ressources.

2.3 Hote (node-exporter)

node-exporter (job `node-exporter`) expose les metriques de la machine hote : CPU, memoire disponible, espace disque restant, charge systeme (load average).

2.4 Bases de donnees (exporters)

Base	Exporter	Cible (job)	Donnees
PostgreSQL metier (<code>healthai</code>)	postgres-exporter	<code>postgres-metier</code>	<code>pg_up</code> , connexions, commits/rollbacks, taille des bases
PostgreSQL auth (<code>auth_db</code>)	postgres-exporter-auth	<code>postgres-auth</code>	idem
MongoDB (<code>reco_fitness</code>)	mongodb-exporter	<code>mongodb</code>	<code>mongodb_up</code> , connexions, compteurs d'operations, taille des collections

2.5 Logs centralises (Promtail vers Loki)

Promtail lit la sortie standard et d'erreur de **tous les conteneurs** via le socket Docker et les pousse vers Loki. Les logs sont etiquetes par `container` , `stream` (stdout/stderr) et `service` , avec une retention de 7 jours. Ils se consultent dans Grafana (Explore ou le panneau "Logs recents" du tableau de bord). Loki ne se requete pas directement, il passe par Grafana.

3. Les 3 alertes Prometheus

Definies dans `monitoring/prometheus/alerts.yml` , regroupees et consultables dans Alertmanager. En environnement de demonstration, aucune notification externe n'est configuree (pas d'email ni de Slack) : on consulte les alertes dans l'UI.

Alerte	Condition	Duree	Severite	Sens
<code>CibleInjoignable</code>	<code>up == 0</code>	1 min	critical	Une cible scrappee ne repond plus (service arrete, plante, ou image non instrumentee).
<code>ConteneurCpuEleve</code>	<code>CPU > 0,9</code> coeur	5 min	warning	Un conteneur <code>mspr-*</code> consomme plus de 0,9 coeur en continu.
<code>ConteneurMemoireElevee</code>	<code>RAM > 2 Go</code>	5 min	warning	Un conteneur <code>mspr-*</code> utilise plus de 2 Go de RAM en continu.

Les deux alertes de ressources ne ciblent que les conteneurs dont le nom commence par `mspr-` . Les seuils sont volontairement larges (demonstration), a ajuster selon le materiel.

4. Acces aux outils

Outil	URL	Usage	Identifiants
Grafana	http://localhost:3001	Tableaux de bord (metriques + logs), point d'entree principal	admin / admin par default
Prometheus	http://localhost:9090	Etat des cibles (Status > Targets), test de requetes, regles d'alerte	-
Alertmanager	http://localhost:9093	Alertes actuellement declenchees	-
Loki	via Grafana	Logs centralises (datasource provisionnee dans Grafana)	-

Le mot de passe Grafana se surcharge via `GRAFANA_ADMIN_PASSWORD` dans `.env`. Le tableau de bord "MSPR HealthAI - Vue stack" (dossier "MSPR HealthAI") affiche les cibles UP, la disponibilite par job, le CPU et la memoire par conteneur, et le flux de logs.

5. Conduite a tenir (runbook)

Reflexe general quand une alerte se declenche : identifier le service concerne (label `name` ou `job` dans l'alerte), puis suivre les etapes ci-dessous.

5.1 Verifier l'etat des conteneurs

```
docker compose -f docker-compose.yml -f docker-compose.monitoring.yml ps
```

On regarde le statut et la sante (colonne `STATUS`, mention `healthy` / `unhealthy`) du service vise.

5.2 Consulter les logs

```
docker compose logs --tail=100 <service>
```

Ou via Grafana (Explore, datasource Loki, filtre sur le label `container`) pour correler avec les autres services sur la meme periode.

5.3 Regarder Grafana

Ouvrir le tableau de bord "MSPR HealthAI - Vue stack" et observer la courbe CPU / memoire du conteneur et l'evolution dans le temps (pic ponctuel ou derive continue).

5.4 Par alerte

- CibleInjoignable** : verifier que le conteneur tourne (`ps`). S'il est arrete ou `unhealthy`, lire ses logs puis le relancer (`docker compose up -d <service>` ou `restart`). Si le conteneur tourne mais reste injoignable, verifier que son endpoint de metriques est expose (typiquement une image applicative pas encore instrumentee, voir 2.1) : dans ce cas l'application fonctionne, seule la collecte manque.

- **ConteneurCpuEleve** : verifier la charge reelle dans Grafana. Pic transitoire (ex. ETL ou inference IA) : non bloquant, l'alerte retombe seule. Derive durable : inspecter les logs, envisager une limite de ressources (voir la configuration "performance" dans `CONFIGS.md`).
- **ConteneurMemoireElevee** : meme demarche. Surveiller une fuite memoire (montee continue sans redescende) ; au besoin redemarrer le conteneur et suivre la tendance.

Tant que la cause n'est pas levee, l'alerte reste active dans Alertmanager ; elle disparaît automatiquement une fois la condition revenue a la normale.

6. Lancer la stack de supervision

L'overlay se combine au compose applicatif sans le modifier.

```
# Mode prod (images GHCR)
docker compose -f docker-compose.yml -f docker-compose.monitoring.yml up -d

# Mode dev (build local, necessaire pour voir les metriques applicatives a jour)
docker compose -f docker-compose.dev.yml -f docker-compose.monitoring.yml up -d --build
```

Arret de la supervision seule, sans toucher a l'application :

```
docker compose -f docker-compose.monitoring.yml down
```

Apres demarrage, verifier dans Prometheus (<http://localhost:9090>, Status > Targets) que les cibles passent a l'etat UP, puis ouvrir Grafana (<http://localhost:3001>).